

RiseForIt / Stryve — Scoring Engine Implementation Work Order

For the coding agent (Fable-5 in VS Code) operating on D:\Jasons Web

Version 2.0 — 2026-07-13 — constants FILLED, Railway deployment included

READ THIS FIRST — Rules of engagement (non-negotiable)

You are implementing a server-authoritative scoring engine for a fitness app. Follow these rules exactly. The owner is away; there is no one to ask, so when in doubt, **prefer the safe/reversible choice and leave a `// TODO(owner): comment`** rather than guessing.

1. **Additive only. Never destructive.** No `DROP TABLE`, no `DROP COLUMN`, no `DELETE` of user data, no renaming existing columns. All schema changes are `CREATE TABLE IF NOT EXISTS` or `ALTER TABLE ... ADD COLUMN IF NOT EXISTS`, added inside the existing `ensureSchema()` startup pattern (this repo has no migration framework — match that pattern).
2. **Constants are already filled — use them.** Every tunable lives in `core/scoringConstants.js`, which ships **alongside this doc, fully populated** with real values tagged by confidence (`HIGH / MEDIUM / DESIGN / VERIFY-PRECISION`). Drop it in as-is. **Do not invent or “improve” any scientific number.** The only two entries marked as optional accuracy upgrades — exact DOTS coefficients and the McCulloch age table — already have a working default (allometric scaling); leave them as-is unless the owner supplies the official coefficient block. Never bury a magic number in engine logic — if you need one, it goes in the constants file.
3. **Feature-flag the whole thing.** All new scoring runs behind `SCORING_ENGINE_V2` (env var, default `off`). While off, the live app behaves exactly as today. Nothing user-facing changes until the owner flips the flag — locally **and** on Railway (see Task 11).
4. **One task = one commit.** Each numbered task is independently shippable. Commit after each with the message prefix shown. Keep `SCORING_CHANGELOG.md` at repo root updated as you go so the owner can read what changed.
5. **Match existing style.** Backend is raw Node http (not Express) — route modules are plain functions dispatched by URL prefix in `server.js`. Frontend is vanilla-JS IIFE modules on `window.*`. Postgres via Neon (`core/db.js getPool()`). Mirror the surrounding code; no frameworks, build steps, or TypeScript.
6. **Write tests for every pure function** (Task 10), house style of `tests/trainingEngine.*.test.js`.
7. **Reconcile reference code with reality.** The code below is correct in structure but written from an audit, not the live files. Before pasting, open the referenced file and match exact existing names. Where this doc and live code disagree on a name, **live code wins** — adapt and note it in the changelog.
8. **Self-verify each task** against its Acceptance Criteria, run tests, record pass/fail in the changelog. Never mark a task done with failing tests.

CONTEXT — current state (so you don't need the full audit)

- **Two disconnected scoring systems today:** the six-axis spider graph runs 100% client-side off localStorage (`js/identity-engine.js`); a monthly points leaderboard computes separately from Postgres (`core/leaderboardRoutes.js` → `scoreUserForMonth`); a third path approximates axes at `js/leaderboard.js:544`. **We replace all three with one server-side engine.**
- **Raw data is already in Postgres and is good:** `app_training_workouts` (sets jsonb, `duration_ms`, `timer_started_at/ended_at`), `app_training_lift_history` (Epley `best_estimated_1rm_lb` — already computed, unused by scoring), `app_daily_checkins` (jsonb/day: macros, mealsOnPlan, weightLb, sleepHours, mood, stress, circumferences), `app_health_daily` (steps/active_minutes/sleep_minutes/distance_meters/gym_visit/wake_result + sources provenance jsonb), `app_progress_photos`.
- **Gaps you are fixing:** sex not stored (blocks normalization); scores never persisted server-side; edits overwrite with no audit; ranks instantaneous (no sustain); provenance stored but ignored; Epley e1RM ignored by strength axis; cardio is a binary 8,000-step credit; Strava HR/distance discarded; Fitbit heart scope never requested; timezone inconsistent (check-in day-key UTC vs health server-local); zero scoring tests.

TARGET ARCHITECTURE (one paragraph)

A new server module `core/scoringEngine.js` computes all six axes (0–100) from Postgres via pure functions, reading every tunable from `core/scoringConstants.js`. Scores + a full event ledger persist to three new tables. A nightly job plus on-write hooks keep scores fresh. The client `identity-engine.js` becomes a display-only renderer that fetches authoritative scores from `GET /api/score`, falling back to a clearly-labeled local estimate only when offline. Everything is gated behind `SCORING_ENGINE_V2`, in both environments.

TASK 1 — Schema migrations (commit: feat(scoring): v2 schema)

Add to the appropriate `ensureSchema()` in `core/trainingRoutes.js` (profiles) and a new `ensureScoringSchema()` called at startup (wire into `server.js` next to the other schema ensures).

```
-- 1a. Normalization inputs (the blocker fix).
ALTER TABLE app_training_profiles ADD COLUMN IF NOT EXISTS sex text;
-- 'male' | 'female' | 'unspecified'
ALTER TABLE app_training_profiles ADD COLUMN IF NOT EXISTS dob date;
-- preferred over age int
ALTER TABLE app_training_profiles ADD COLUMN IF NOT EXISTS timezone text;
-- IANA, e.g. 'America/New_York'
```

-- 1b. Authoritative daily score snapshot (enables sustain gates + honest decay + history).

```
CREATE TABLE IF NOT EXISTS app_score_snapshots (  
  user_id uuid NOT NULL REFERENCES app_users(id) ON DELETE CASCADE,  
  day date NOT NULL,  
  strength numeric(5,2) NOT NULL DEFAULT 0,  
  cardio numeric(5,2) NOT NULL DEFAULT 0,  
  consistency numeric(5,2) NOT NULL DEFAULT 0,  
  nutrition numeric(5,2) NOT NULL DEFAULT 0,  
  recovery numeric(5,2) NOT NULL DEFAULT 0,  
  progress numeric(5,2) NOT NULL DEFAULT 0,  
  rank text NOT NULL DEFAULT 'peasant',  
  caste text,  
  weeks_at_rank int NOT NULL DEFAULT 0,  
  normalized boolean NOT NULL DEFAULT false,    -- false = self-improvement-  
only (sex missing)  
  computed_at timestamptz NOT NULL DEFAULT now(),  
  PRIMARY KEY (user_id, day)  
);
```

-- 1c. Append-only event ledger (explainability + audit + anomaly flags).

```
CREATE TABLE IF NOT EXISTS app_score_events (  
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),  
  user_id uuid NOT NULL REFERENCES app_users(id) ON DELETE CASCADE,  
  created_at timestamptz NOT NULL DEFAULT now(),  
  axis text NOT NULL,  
  delta numeric(6,2),  
  reason_code text NOT NULL,  
  provenance text,                                -- 'device' / 'self_report' /  
'implausible'  
  trust_multiplier numeric(3,2),  
  source_ref text,  
  detail jsonb NOT NULL DEFAULT '{}'::jsonb  
);  
CREATE INDEX IF NOT EXISTS idx_score_events_user_time ON app_score_events  
(user_id, created_at DESC);
```

-- 1d. Edit audit (fixes upsert-in-place destroying history).

```
CREATE TABLE IF NOT EXISTS app_edit_audit (  
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),  
  user_id uuid NOT NULL REFERENCES app_users(id) ON DELETE CASCADE,  
  created_at timestamptz NOT NULL DEFAULT now(),  
  table_name text NOT NULL,  
  row_key text NOT NULL,  
  field text,  
  old_value jsonb,  
  new_value jsonb,  
  actor text NOT NULL DEFAULT 'user'  
);
```

```
-- 1e. Provenance on check-ins (currently none).
ALTER TABLE app_daily_checkins ADD COLUMN IF NOT EXISTS sources jsonb NOT
NULL DEFAULT '{}'::jsonb;

-- 1f. Wake timestamps for sleep-regularity (today only the day is stored).
ALTER TABLE app_health_daily ADD COLUMN IF NOT EXISTS wake_at timestamptz;
ALTER TABLE app_health_daily ADD COLUMN IF NOT EXISTS sleep_start_at
timestamptz;
```

Onboarding write path: in POST /api/training/onboarding (validatePlanInputs), accept and persist sex (male|female|unspecified) and dob. Add a one-time, non-blocking modal for existing users to fill sex/dob. Derive age from dob at compute time; keep age int as fallback when dob is null.

Acceptance criteria: app boots; all tables/columns exist; existing flows unchanged; onboarding stores sex/dob; nothing destructive.

TASK 2 — Drop in core/scoringConstants.js (commit: feat(scoring): constants module)

The complete, filled constants file ships with this hand-off (scoringConstants.js). Place it at core/scoringConstants.js exactly as delivered. It is already populated with real values, each tagged:

- **HIGH** — settled/canonical (Mifflin–St Jeor, WHO 150–300, protein 1.6–2.2 g/kg, 7–9h sleep, MET table, 0.5–1% fat loss, allometric 0.67 exponent, VO2max ~10%/decade).
- **MEDIUM** — established but approximate (VO2max percentile anchors, female:male per-lift ratios, muscle-gain rates, bodyweight-multiple strength standards).
- **DESIGN** — owner decisions (axis blend 60/40, trust multipliers, windows, decay rates, rank gates, plausibility thresholds).
- **VERIFY-PRECISION** — exact DOTS coefficients and the McCulloch age table. **These already have a working default (allometric scaling + gentle per-year age handicap), so they are NOT blockers.** Leave strength.dotsUpgrade.enabled=false unless the owner supplies the official coefficient block.

Do not edit the numbers. If the engine needs a value not present, add it to this file (with a confidence tag) rather than inlining it.

Acceptance criteria: require('./core/scoringConstants') loads; every axis in the engine reads from it; grep shows no magic scoring numbers in scoringEngine.js.

TASK 3 — core/scoringEngine.js (commit: feat(scoring): engine core)

Pure, testable functions; zero DB imports. Structure below; adapt field access to the gathered shape.

```
// core/scoringEngine.js
'use strict';
const C = require('./scoringConstants');
const clamp = (v, lo, hi) => Math.max(lo, Math.min(hi, v));
const AXES = ['strength', 'cardio', 'consistency', 'nutrition', 'recovery', 'progress'];

function ageFrom(profile, today) {
  if (profile.dob) return Math.floor((today - new Date(profile.dob)) /
(365.25 * 864e5));
  return Number(profile.age) || null;
}
function trustMultiplier(p) {
  return p === 'device' ? C.trust.device : p === 'self_report' ?
C.trust.self_report : C.trust.implausible;
}

// STRENGTH standard: allometric-adjusted e1RM mapped to 0-100 via
bodyweight-multiple bands.
// Returns { standard:0..100|null, normalized:bool }. sex missing -> self-
only.
function normalizeStrength(bestE1rmByLift, profile) {
  const sex = profile.sex;
  if (sex !== 'male' && sex !== 'female') return { standard: null,
normalized: false };
  const bwLb = Number(profile.bodyweightLb) || 0;
  if (bwLb <= 0) return { standard: null, normalized: true };
  const bands = C.strength.bwMultipleStandards[sex];
  const ap = C.strength.anchorPoints;
  let sum = 0, n = 0;
  for (const lift of C.strength.mainLifts) {
    const e1 = bestE1rmByLift[lift];
    if (!e1 || !bands[lift]) continue;
    const mult = e1 / bwLb; // relative
strength
    const b = bands[lift];
    // piecewise-linear map novice->intermediate->advanced->elite onto
anchorPoints
    let pts;
    if (mult <= b.novice) pts = (mult / b.novice) * ap.novice;
    else if (mult <= b.intermediate) pts = ap.novice + (mult -
b.novice)/(b.intermediate-b.novice)*(ap.intermediate-ap.novice);
    else if (mult <= b.advanced) pts = ap.intermediate + (mult -
b.intermediate)/(b.advanced-b.intermediate)*(ap.advanced-ap.intermediate);
    else if (mult <= b.elite) pts = ap.advanced + (mult -
```

```

b.advanced)/(b.elite-b.advanced)*(ap.elite-ap.advanced);
    else pts = ap.elite;
    sum += clamp(pts, 0, 100); n += 1;
}
if (!n) return { standard: null, normalized: true };
// age handicap
const age = ageFrom(profile, new Date());
let ageAdj = 1;
if (age && age > 40) ageAdj = 1 + C.age.strengthPerYearOver40 * (age - 40);
// credit older lifters
return { standard: clamp((sum / n) * ageAdj, 0, 100), normalized: true };
}

function selfImprovement(current, baseline) {
    if (!baseline || baseline <= 0) return 50;
    return clamp(50 + (current / baseline - 1) * 200, 0, 100); // +25% over
baseline ~ full
}
function blend(standard, self) {
    return standard == null ? self : C.blend.standardWeight * standard +
C.blend.selfWeight * self;
}
function applyDecayOrPause(prev, axis, idleDays, pauseActive, seed) {
    if (pauseActive) return prev;
    const d = C.decay[axis];
    return idleDays <= d.graceDays ? prev : Math.max(seed, prev - d.perDay *
(idleDays - d.graceDays));
}
function computeRank(axisScores, weeksAtCandidate) {
    const g = C.ranks, v = AXES.map(a => axisScores[a]);
    const avg = v.reduce((x, y) => x + y, 0) / v.length, min = Math.min(...v),
spike = Math.max(...v);
    let rank = 'peasant';
    for (const [id, gate] of Object.entries(g)) {
        const byAvg = avg >= gate.avg && min >= (gate.floor || 0);
        const byPierce = !gate.noPierce && spike >= (gate.spike || 999);
        const sustainOk = (weeksAtCandidate[id] || 0) >= (gate.sustainWeeks ||
0);
        if ((byAvg || byPierce) && sustainOk) rank = id;
    }
    let caste = null;
    if (rank === 'specialist') for (const [id, fn] of Object.entries(C.castes))
if (fn(axisScores)) { caste = id; break; }
    return { rank, caste };
}

// Implement computeStrength/Cardio/Consistency/Nutrition/Recovery/Progress:
// gather signal -> normalize -> blend with self -> apply trust multiplier ->
// window/decay/pause -> push explanatory events. Read ONLY from C.*.

```

```
module.exports = { AXES, ageFrom, trustMultiplier, normalizeStrength,
selfImprovement, blend, applyDecayOrPause, computeRank };
```

Also write **core/scoringGather.js**: `gatherUserScoringInputs(db, userId, sinceDate)` queries `app_training_lift_history`, `app_health_daily`, `app_daily_checkins`, `app_training_workouts`, `app_training_profiles` and returns one plain object the engine consumes. All SQL lives here; the engine stays pure.

Acceptance criteria: engine has zero DB imports; every number comes from constants; `computeRank` respects sustain weeks; sex-missing users get `normalized=false` + a valid self score; Task 10 tests pass.

TASK 4 — Persistence + recompute triggers (commit: feat(scoring): recompute + persist)

- **Nightly job:** a real recompute pass (boot + every 6h) in `server.js`, **separate from the grocery-scraper interval at `server.js:2535`**. Per active user: `gather` → `compute` → upsert one `app_score_snapshots` row for today; append `app_score_events`; increment `weeks_at_rank` when rank held ≥ 7 days. **Single-instance guard: wrap the job in a Postgres advisory lock (`pg_try_advisory_lock`) so it never double-runs if Railway starts more than one replica** (see Task 11).
- **On-write hook:** after `POST /api/training/log`, `POST /api/training/checkin`, and health syncs, enqueue a lightweight recompute for that user (responsiveness).
- **New endpoint `GET /api/score`** (auth required): latest snapshot + recent events for the signed-in user. This is what the client renders.
- All of the above **only runs when `SCORING_ENGINE_V2` is on**; otherwise legacy behavior is untouched.

Acceptance criteria: flag on → logging a workout updates the snapshot within one request cycle; nightly writes one row/user/day; `GET /api/score` returns axes + rank + explanations; flag off → zero behavior change; advisory lock prevents double-run.

TASK 5 — Client becomes display-only (commit: feat(scoring): client renders server score)

- `js/identity-engine.js`: when `SCORING_ENGINE_V2` is on, `refresh()` fetches `GET /api/score` and renders into `ovIdentityStats`. Keep the local growth model **only** as an offline fallback, tagged `stats.__estimated = true` so it's visibly non-authoritative.
- `js/leaderboard.js` and `js/overview-identity.js` read the same server axes — **remove the divergent `js/leaderboard.js:544` approximation** for the signed-in user.

Acceptance criteria: signed-in radar matches GET /api/score exactly; offline shows a clearly-labeled estimate; the browser cannot change the authoritative score.

TASK 6 — Trust ladder wiring (commit: feat(scoring): trust ladder)

- Map app_health_daily.sources per metric: strava|fitbit|gps|alarm → device (×1.0); manual/in-app → self_report (×0.7).
- Tag lift sessions device when the workout's timer window falls within a gym-geofence visit (gym_visit=true), else self_report.
- Write sources on app_daily_checkins going forward (manual → manual).
- Apply the multiplier inside each compute*; record provenance + trust_multiplier on every event.

Acceptance criteria: a Strava-synced run out-tiers the same minutes typed in; events show the multiplier; honest self-report never scores zero.

TASK 7 — Integrity / anti-gaming (commit: feat(scoring): integrity)

- **Edit audit:** before overwriting in POST /api/training/checkin and /api/health/manual, read the old row and insert app_edit_audit (old→new).
- **Rate limits:** per-user, per-endpoint write throttle on scoring-relevant POSTs (in-memory token bucket).
- **Anomaly flags → app_score_events (reason_code='flag_*', trust 0 on the offending delta):** e1RM jump > C.strength.implausibleWeeklyGainPct/week; nutrition calories impossible vs 7-day weight trend; GPS teleport/accuracy spoof; **plausibility timer** — compare duration_ms vs projected (prescribed restSec + estimateExerciseMinutes()) at core/trainingEngine.js:1594); void the session's strength credit if actual < C.plausibility.voidBelowFractionOfProjected, **unless** submitted within lateSubmissionGraceHours (forgot-to-submit protection).
- **Timezone fix:** persist per-user timezone; compute the day-boundary once from it everywhere (reconcile check-in-UTC vs health-server-local).

Acceptance criteria: editing a past day leaves an audit row; a 10-second “45-minute” workout is voided; a genuine late submit still counts; impossible 1RM leaps flagged not scored.

TASK 8 — Cardio rebuild + integrations (commit: feat(scoring): cardio v2)

- Replace the binary 8,000-step credit (js/identity-activity.js:121, js/main.js:1447, core/healthRoutes.js:351 dayPoints) with **graded minutes-at-intensity → 0-100** from C.cardio.whoMinutesAnchors + C.cardio.metValues; steps demoted to stepsFallbackCapPoints.

- **Add the Fitbit heart scope** (`healthRoutes.js:266`) and store resting HR / HR zones / VO2max when available. **This requires the Fitbit app registration to include heart — see Task 11.**
- **Stop discarding Strava HR/distance/type** (`healthRoutes.js:236`) — persist for intensity grading (extend `app_health_daily` or add `app_health_activities`).
- **Dedup:** same session in-app + via Strava/Fitbit (overlapping time window) counts once (prefer device-verified).

Acceptance criteria: cardio reflects intensity not just steps; HR used when present; a run synced twice counts once.

TASK 9 — Ranks consolidation (`commit: feat(scoring): ranks unified`)

- Keep **one** gate table (`C.ranks`) on the authoritative path. Reconcile the CASTES predicate table (`leaderboard.js:450`) and UI-copy numbers (`:185`) to match, or mark them cosmetic.
- Drive rank off `app_score_snapshots` with **sustain-weeks** (King = 8 weeks). Keep 3-day demotion stickiness as UX polish.
- Fabricated leaderboard population stays **cosmetic only** — never influences a real rank.

Acceptance criteria: a momentary King-level spike does NOT promote until sustained 8 weeks; one gate table everywhere; unlimited holders.

TASK 10 — Tests (`commit: test(scoring): engine coverage`)

Plain Node test files in `tests/`. Cover at minimum: - `normalizeStrength`: female vs male at same absolute load → female scores higher for equal bodyweight (fairness principle); sex missing → `normalized=false` + self score. - Best-recent bank: 350 then 320 → strength does not drop (owner's example). - Decay vs pause: idle past grace decays; pause freezes; rebuild from seed floor. - Trust ladder: `device > self_report`; implausible → 0; honest self-report never zero. - Rank sustain: King requires 8 weeks; instantaneous spike does not promote. - Plausibility: 10s/45min workout voided; late-but-plausible submit counts. - Cardio: graded intensity beats steps-only; steps capped at `stepsFallbackCapPoints`.

Acceptance criteria: all tests pass; determinism preserved.

TASK 11 — Railway deployment + environment parity (`commit: chore(scoring): railway env`)

The app runs on **Railway** against **Neon Postgres**. Fixes must work in production, not just locally. Do this without exposing secrets.

A. Environment variables — set in the Railway service (Variables tab), matching local .env:

Variable	Purpose	Local	Railway
DATABASE_URL	Neon connection string (*.neon.tech → serverless driver via core/db.js)	already set	verify set & points at prod Neon
SCORING_ENGINE_V2	Master feature flag	false	false at launch — owner flips to true after verifying
FITBIT_CLIENT_ID / FITBIT_CLIENT_SECRET	Fitbit OAuth	set	set
FITBIT_SCOPES	must now include heart (Task 8)	activity sleep profile heart	same
STRAVA_CLIENT_ID / STRAVA_CLIENT_SECRET	Strava OAuth	set	set
OAUTH_REDIRECT_BASE	callback base URL — differs by env (localhost vs Railway domain)	localhost	Railway public domain
DEFAULT_TZ	fallback timezone for day-boundary (Task 7)	America/New_York	same
NODE_ENV	production on Railway	development	production

- **Do not hardcode any of these.** Read from process.env (match how core/db.js already does). Add a startup check that logs (not throws) any missing scoring-relevant var so the owner sees it in Railway logs.
- **Fitbit heart scope:** the Fitbit developer app registration itself must be updated to request heart, and the Railway FITBIT_SCOPES set to match — otherwise the new HR pull returns 403 in prod. Note this in the changelog for the owner to confirm in the Fitbit dashboard.
- **OAuth redirect URIs:** the Strava and Fitbit app registrations must list the **Railway production callback URL** in addition to localhost, or prod OAuth breaks. Flag for owner action.

B. Schema on deploy: the ensureSchema() / ensureScoringSchema() calls run at boot, so Task 1's migrations auto-apply on the next Railway deploy. **Verify in the Railway deploy logs** that the new tables/columns were created against the prod Neon DB, and record it in the changelog.

C. Nightly job & multiple replicas: if Railway ever runs >1 replica, a plain `setInterval` recompute would double-run. The `pg_try_advisory_lock` guard from Task 4 must wrap the job so only one instance computes per tick. Confirm the lock is released on completion.

D. Parity check: produce a short `DEPLOYMENT.md` listing every env var, where it's set (local vs Railway), and the two owner-action items (Fitbit scope in the Fitbit dashboard, OAuth redirect URIs). This is what the owner reviews before flipping `SCORING_ENGINE_V2` to true.

Acceptance criteria: app boots on Railway with all vars present; deploy logs show the new schema applied to prod Neon; OAuth still works in prod; nightly job single-runs under the advisory lock; `DEPLOYMENT.md` written; flag stays false until the owner flips it.

DEFINITION OF DONE (whole project)

- ☐ All 11 tasks committed with the shown prefixes; `SCORING_CHANGELOG.md` current.
 - ☐ `SCORING_ENGINE_V2` off → app behaves exactly as before (verify a normal user flow), locally and on Railway.
 - ☐ `SCORING_ENGINE_V2` on (in a staging check) → `GET /api/score` returns authoritative axes + rank + explanations; browser cannot alter them.
 - ☐ sex/dob captured; normalization runs when present, self-only when absent.
 - ☐ No destructive migration; no dropped data; edit audit populated.
 - ☐ All scoring numbers read from the shipped `scoringConstants.js`; no invented standards in logic.
 - ☐ Tests green.
 - ☐ `DEPLOYMENT.md` written; Railway env vars verified; schema confirmed applied to prod Neon in deploy logs.
 - ☐ `SCORING_CHANGELOG.md` summarizes what changed, what's behind the flag, and the owner-action items (Fitbit heart scope, OAuth redirect URIs, flip the flag).
-

GUARDRAILS — what NOT to do

- Do **not** turn `SCORING_ENGINE_V2` on in production. Leave it false in Railway; the owner flips it after reviewing `DEPLOYMENT.md`.
- Do **not** edit the values in `scoringConstants.js` or invent standards. The two `VERIFY-PRECISION` items already have working defaults — leave `DOTS` disabled unless the owner supplies official coefficients.
- Do **not** delete/rewrite the grocery scraper, training generator, or any non-scoring system.
- Do **not** move progress photos or other large data in this pass (separate future task).
- Do **not** hardcode secrets or environment-specific URLs — everything through `process.env`.

- Do **not** collapse the two day-boundary conventions by guessing a timezone — read the stored `timezone`, default to `DEFAULT_TZ` with a `// TODO(owner)` only if truly absent.
 - If any referenced file/line/column name doesn't match reality, **stop, match the live code, and note it** — do not force this doc's names.
-

OPTIONAL PRECISION UPGRADES (not blockers)

The engine ships fully functional on the filled constants. Two entries can later be swapped for maximum textbook precision, each a drop-in that changes only numbers, not structure: (1) **exact DOTS/IPF-GL coefficients** in place of the allometric strength normalizer (set `strength.dotsUpgrade.enabled=true` after pasting the official men's/women's coefficient block from the OpenLifter source); (2) the **McCulloch age-coefficient table** in place of the linear age handicap. A separate cited-sources document may follow to attach references and tighten the MEDIUM-tagged anchors — but nothing in this build waits on it.